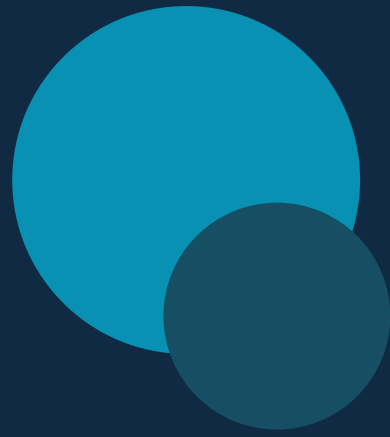




LLM Memory Service

Complete Architecture, Classification, Storage, Retrieval, and Operations Guide

Memory-management MVP for an elderly home assistant

Repository state reviewed: 23 September 2026

Service API version: 0.14.0 | Database: PostgreSQL + pgvector | Local AI: Ollama

Contents

This guide reflects the implemented repository rather than a conceptual target. It separates memory behavior from optional reply-model behavior.

Contents	2
1. Executive summary	4
Implemented capabilities	4
Explicit scope boundary	4
2. System architecture	5
Architectural pattern	5
3. End-to-end interaction lifecycle	6
Recommended asynchronous path	6
Ordering and idempotency	6
4. How the system decides long-term, short-term, episode, or discard	7
Two-stage decision	7
Policy precedence	7
5. Important labels and what they mean	8
Reading a typical episode decision	8
6. Long-term profile memory	9
Storage model	9
Consolidation algorithm	9
Correction example	9
7. Short-term/session memory and sliding conversation window	10
Temporary memory	10
Conversation sliding window	10
8. Episodic memory	11
Example	11
9. Database schema and storage responsibilities	12
Migration history	12
Isolation rules	12
10. Persistent embedding pipeline	13
What is embedded	13
Why structured fact text is used	13

Failure behavior	13
11. Retrieval algorithm	14
Profile retrieval stages	14
Candidate formation	14
Other context sources	14
12. Context package consumed by the assistant	15
Prompt assembly	15
Memory references and observability	15
13. Safety, provenance, and emergency behavior	16
Evidence guards	16
Protected information	16
Emergency orchestrator	16
14. API surface	17
Swagger UI	17
15. Installation and one-command operation	18
Prerequisites	18
First-time setup	18
Recommended memory-only demo	18
Optional real chat mode	18
16. How to use and test the system	19
Memory-only commands	19
Feature-covering manual sequence	19
Retrieval checks	19
17. Performance and evaluation evidence	20
Interpretation	20
Current automated verification	20
Model evaluation records	20
18. Configuration reference	21
19. Known limitations and production gaps	22
20. Repository map and inspection basis	23
Recommended demo narrative	23
Final mental model	23

1. Executive summary

The system is a standalone memory microservice. It receives user interactions, extracts structured memory candidates with an Ollama model, validates them with deterministic policy, stores them in the correct lifetime scope, and later builds a bounded context package for an assistant LLM.

Key design rule

Lifetime and sensitivity are independent axes. A health statement is not automatically long-term: a current headache can be session memory, a medication routine can be profile memory, and a past fall can be an episode.

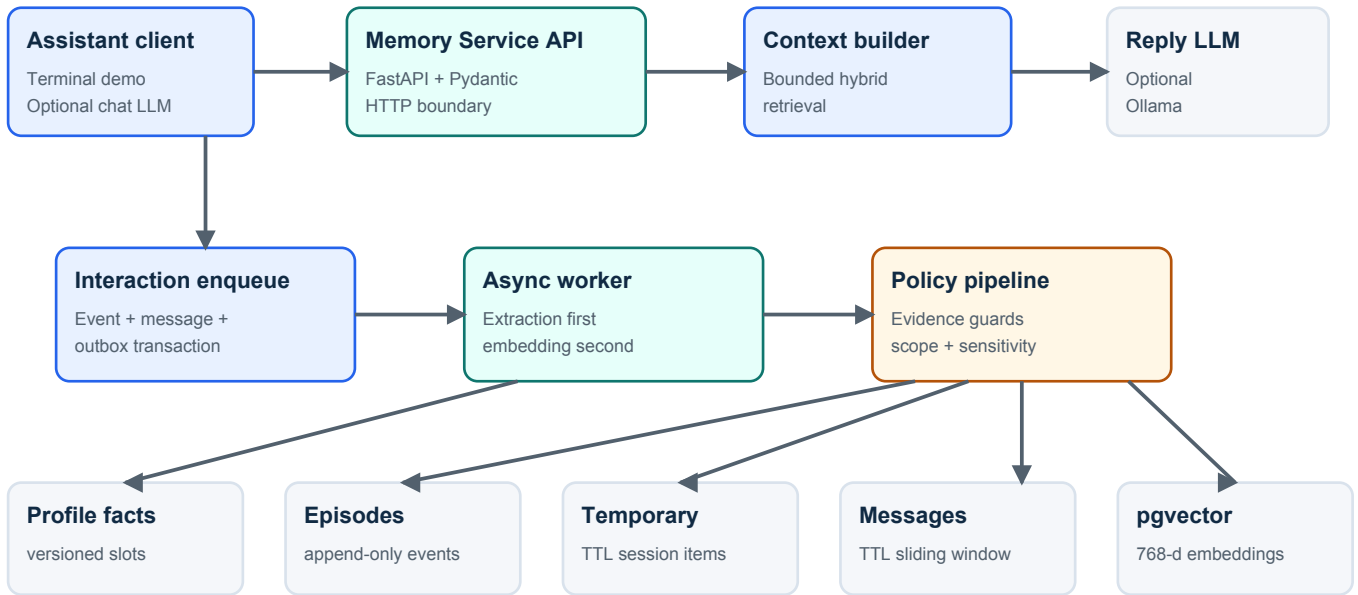
Implemented capabilities

- Versioned long-term profile facts in PostgreSQL.
- Independently expiring short-term/session memories.
- Append-only episodic memory for bounded past events.
- TTL-based conversation history with message and token bounded sliding-window retrieval.
- Ollama semantic extraction followed by deterministic evidence, safety, routing, and consolidation policy.
- Asynchronous ingestion through a durable PostgreSQL outbox and worker.
- Persistent 768-dimensional profile embeddings in pgvector with an HNSW cosine index.
- Hybrid lexical, alias, semantic, recency, confidence, and pinned-category retrieval.
- Idempotent events/messages, session ownership checks, deletion endpoints, and candidate audit records.
- A bounded emergency detector whose phone action is simulation-only.

Explicit scope boundary

STT and TTS are not part of this repository. The chat client and reply LLM are development harnesses. The production deliverable here is the memory API, worker, persistence model, retrieval layer, and demo tooling.

2. System architecture

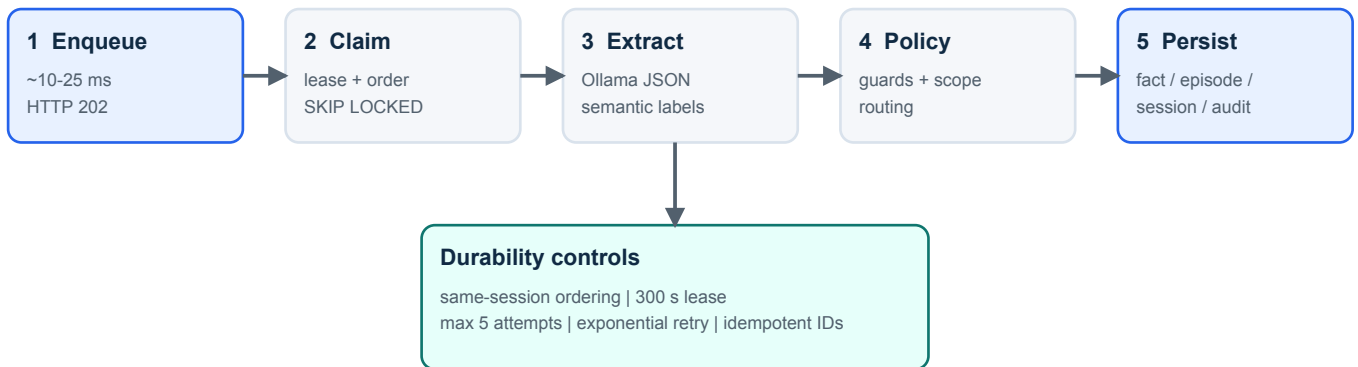


Layer	Implementation	Responsibility
Client/orchestrator	conversation_orchestrator.py and terminal scripts	Requests context, optionally calls a reply LLM, then enqueues the interaction.
Memory API	FastAPI + Pydantic	Validation, ownership, idempotency, storage endpoints, and context construction.
Extraction	Ollama gemma4:12b by demo default	Returns structured semantic labels; does not directly choose the database destination.
Policy	Deterministic Python v3	Verifies evidence, separates scope from sensitivity, assigns TTL, and rejects unsafe content.
Persistence	PostgreSQL + SQLAlchemy + Alembic	Stores events, audit candidates, facts, episodes, temporary items, messages, jobs, and vectors.
Semantic retrieval	embeddinggemma:latest + pgvector	Embeds active profile facts and queries; ranks by cosine similarity.
Worker	PostgreSQL outbox consumer	Processes extraction first and durable embedding jobs second.

Architectural pattern

This is a modular memory microservice with a transactional outbox. Internally it remains one deployable Python service plus a worker, which avoids unnecessary service fragmentation while still keeping slow model work off the conversational request path.

3. End-to-end interaction lifecycle



Recommended asynchronous path

Step	What happens	Durability/performance consequence
1. Context	POST /v1/context:build with user_id, session_id, and current query.	Returns only relevant, bounded memory; does not call the reply LLM.
2. Reply	Optional orchestrator sends context + recent history + user text to the chat model.	Chat latency is mainly model prompt evaluation and token generation.
3. Enqueue	POST /v1/interactions:enqueue writes the raw event, user message, context snapshot, and outbox job in one transaction.	Returns HTTP 202 without waiting for extraction.
4. Extract	Worker claims the oldest eligible job for a session and calls the Ollama extractor.	A slow extractor does not block the already-produced assistant reply.
5. Route /store	Evidence guards and policy select profile, episode, session, or discard; audit candidate is persisted.	All memory writes are traceable to an event.
6. Embed	Active profile fact write queues an embedding job; worker stores the vector.	Lexical/alias retrieval remains available before the vector is ready.

Synchronous endpoint

POST /v1/interactions:process is retained for diagnostics. It waits for the extraction model and can take several seconds; it should not be the normal chat path.

Ordering and idempotency

event_id and message_id are idempotency keys. A repeated ID with the same payload returns the existing result; reuse with changed content produces a conflict. Outbox jobs from the same user/session preserve creation order, while PostgreSQL workers use FOR UPDATE SKIP LOCKED to avoid cross-worker blocking.

4. How the system decides long-term, short-term, episode, or discard

Two-stage decision

The Ollama extractor identifies the meaning of each explicit proposition. The deterministic policy then decides the authoritative scope. This prevents the generative model from directly writing arbitrary memory.

Semantic condition	Resulting scope	Example
Stable profile_fact, preference, or habit; persistent lifetime	profile	Her sabah sade Türk kahvesi içerim.
Today, tomorrow, or current-session intent/state	session	Bugün öğleden sonra yürüyüşe çıkmak istiyorum.
Bounded past event or speech_act=episode	episode	Dün banyoda düştüm.
Question, greeting, courtesy, device command, memory-control instruction, bare fragment, unsupported inference	discard	Bugün hava nasıl? / tenis / Televizyonu aç.
Financial identifier or credential	discard	Password, token, or bank identifier.

Policy precedence

- Explicit bounded time wins over a stable-looking speech act, so a temporary address or plan remains session memory.
- speech_act=episode wins for a bounded past incident even when temporal_scope is unknown.
- Stable speech acts become profile memory; persistent current state can also become profile when it describes an enduring capability or limitation.
- Unknown or third-party transient states are discarded; a stable fact about an explicitly related person may be profile memory.
- Questions and question clauses never establish a new user fact.
- Literal evidence, numbers, clock times, weekdays, and quoted clauses are checked against the current user message.

No confirmation workflow

The former confirm/reject flow has been removed. Explicit health, location, and emergency-contact statements are automatically stored as user_asserted. Candidate status is auto_applied or ignored. Outbox status pending only means waiting for the worker.

5. Important labels and what they mean

Field	Important values	Meaning
scope	profile, episode, session, discard	Authoritative lifetime and storage destination.
category	communication, dietary_preference, medication, health_event, session, emergency_contact	Model-proposed semantic domain, normalized and guarded by policy.
key	form_of_address, coffee_preference, fall_incident, morning_medication_time	Canonical slot inside a category. Profile uses one active value per slot.
value	literal proposition or structured temporary payload	The stored fact. Model-originated strings must be grounded in the current message.
claim_kind	assertion, question, inferred, ambiguous, contextual_reply	Only evidence-backed assertion or a verified contextual reply can be stored.
subject	user, related_person, third_party, unknown	Who the proposition is about. Prevents third-party state from becoming user memory.
speech_act	profile_fact, preference, habit, intent, current_state, episode, question, device_command	Semantic function used by routing policy.
temporal_scope	persistent, today, tomorrow, current_session, unknown	Controls profile vs temporary lifetime and TTL.
sensitivity	normal, personal, health, emergency_contact, location, financial, credential	Privacy/safety axis; independent from scope.
verification_status	unverified, user_asserted, user_confirmed, caregiver_confirmed, system_verified	Provenance strength. Current automatic protected writes use user_asserted.
memory_type	long_term, short_term, sensitive, discard	Legacy/API-facing decision label. scope is authoritative for storage.
analyzer_source	ollama, rules_guard, rules_fallback, rules	Shows whether the model, a safety guard, or fallback produced the decision.
status	auto_applied, ignored	Whether the candidate was stored or rejected.
consolidation_action	create, unchanged, supersede, not_applicable	How a profile slot relates to existing active facts.

Reading a typical episode decision

```
{
  "memory_type": "sensitive",
  "scope": "episode",
  "category": "health_event",
  "key": "fall_incident",
  "value": "Dün banyoda düştüm",
  "sensitivity": "health",
  "verification_status": "user_asserted",
  "status": "auto_applied",
  "destination": "episodic memory (PostgreSQL)"
}
```

Interpretation: the user explicitly reported a health-sensitive, bounded past incident. Sensitivity is health, lifetime is episode, and the statement is attributed as user_asserted rather than clinically verified.

6. Long-term profile memory

Storage model

Long-term facts live in `memory_facts`. The active uniqueness boundary is `user_id` + `category` + `key`. Every version records sensitivity, verification status, confidence, provenance event, validity times, and an optional `supersedes_id`.

Consolidation algorithm

Stage	Technique	Outcome
Normalize category	Alias mapping: preferences -> preference, habits -> routine, medicines -> medication	Different model wording maps to stable domains.
Normalize key	Known slot aliases and ASCII snake_case	For example, morning_wake_up_time -> morning_wake_time.
Compare slot	Exact canonical match, token Jaccard, SequenceMatcher; threshold 0.65 and ambiguity gap 0.08	Finds one likely existing slot without merging ambiguous candidates.
Protect unrelated preferences	Generic tokens such as preference/style/time do not merge disjoint specific subjects	Pasta preference does not replace coffee preference.
Compare value	Normalized JSON/text; optional terminal-period tolerance for ordinary memories	Equivalent statements become unchanged.
Apply	create, unchanged, or supersede	Only one active slot remains; old versions are retained as history.

Correction example

1. "Her sabah sade Türk kahvesi içerim."
-> profile / dietary_preference.coffee_preference / create
2. "Düzeltilme: Her sabah sütlü Türk kahvesi içerim."
-> same canonical slot / supersede

Result: one active milk-coffee fact; the former fact remains superseded history.

Embedding timing

The profile row and its durable embedding job are committed together. Vector generation happens later in the worker, so the database fact is immediately available to lexical/alias retrieval.

7. Short-term/session memory and sliding conversation window

Temporary memory

Short-term items live in `temporary_memories` and are scoped by `user_id` + `session_id`. Each item has its own category, key, value, `occurred_at`, `expires_at`, sensitivity, and provenance. Multiple independent intentions coexist. Only the same resolved slot is replaced.

Temporal label	Expiry rule	Example
today	End of the current local day in <code>MEMORY_TIMEZONE</code>	Bugün yürüyüşe çıkmak istiyorum.
tomorrow	End of the next local day	Yarın kızımı ziyaret etmek istiyorum.
current_session / unknown transient	<code>occurred_at</code> + <code>MEMORY_TEMPORARY_TTL_MINUTES</code> (default 60, bounded 1-1440)	Şimdi biraz dinlenmek istiyorum.

Expired items are filtered from reads. They are not automatically deleted at the exact TTL boundary; they remain as database history until a deletion/retention process removes them. The session deletion endpoint removes session state, messages, and temporary items.

Conversation sliding window

Conversation messages are a separate short-lived store used for dialogue continuity. The service queries unexpired messages for the same user/session, starts from the newest, applies both limits, then returns the chosen suffix in chronological order.

Control	Default	Behavior
MEMORY_WINDOW_MAX_MESSAGES	10	At most ten recent messages are included.
MEMORY_WINDOW_MAX_TOKENS	2000	Approximate token budget using three characters per token.
MEMORY_CONVERSATION_TTL_HOURS	24	Messages older than the TTL are excluded.
Async safety	enabled	Unprocessed/failed/canceled async user turns and linked assistant replies are excluded from Memory Service context.

Important distinction

Temporary memories represent extracted short-lived state. The sliding window represents raw recent dialogue. A statement may appear in `recent_messages` before or after it becomes a structured temporary memory, but these are separate mechanisms.

8. Episodic memory

Episodes capture bounded past events such as a fall, accident, hospitalization, or surgery. They are stored in `memory_episodes` and do not occupy or replace profile slots.

Property	Implemented behavior
Write model	Append-only. Repeated submissions currently create repeated episodes.
Ownership	User-scoped; <code>session_id</code> is recorded when available.
Time	<code>occurred_at</code> currently uses the source interaction time. Relative phrases such as 'dün' remain in the literal value.
Retention	<code>retention_until</code> may expire an episode; null means no configured expiry.
Retrieval	Bounded lexical/alias/recency/confidence selector, max 10 items and 1000 estimated tokens by default.
Protected gate	Profile health-domain gate is disabled for episode recall so a temporal query such as 'Dün ne olmuştu?' can retrieve a matching health event.
Embeddings	The current pgvector pipeline indexes profile facts only, not episodes.

Example

Input: Dün banyoda düştüm.

Extractor labels:

`claim_kind=assertion, subject=user, speech_act=episode,
temporal_scope=unknown, sensitivity_domain=health`

Policy result:

`scope=episode, category=health_event, key=fall_incident,
sensitivity=health, verification_status=user_asserted`

Later query: `/context Dün ne olmuştu?`

-> episodes contains the stored fall incident.

Known limitation

Episode deduplication is not implemented. Re-entering the same fall event creates multiple append-only rows. This is visible and auditable but can produce duplicate recall.

9. Database schema and storage responsibilities

Table	Purpose	Key behavior
memory_events	Idempotent raw event log	Primary key is caller event_id; source for audit/provenance.
conversation_messages	Recent user/assistant turns	Per-session TTL and parent_message_id linkage.
memory_outbox	Durable extraction queue	One job per event; leases, retries, ordering, terminal states.
memory_candidates	Decision audit	Semantic metadata, scope, sensitivity, consolidation, status, destination reference.
memory_facts	Versioned long-term profile	One active row per user/category/key; old versions superseded.
memory_episodes	Bounded past events	Append-only; occurred_at and optional retention_until.
temporary_memories	Independent session items	One active row per user/session/category/key; finite TTL.
session_states	Explicit structured state and observations	Whole state has an expiry and version.
memory_embedding_jobs	Durable vector work	One job per memory/model with retries and content hash.
memory_embeddings	Persistent profile vectors	Composite key memory_id/model; 768 dimensions; cascade delete.

Migration history

Thirteen Alembic revisions build the schema incrementally: initial events/session/facts, candidate audit, analyzer provenance, semantic metadata, conversation window, consolidation, temporary memories, parent links, outbox, user-asserted health, pgvector embeddings, explicit memory scope + episodes, and removal of confirmation flow.

Isolation rules

- All retrieval is filtered by user_id; session stores are also filtered by session_id.
- Session ownership is checked across session state, messages, and temporary memory.
- A source event used by temporary memory must belong to the same user and session.
- Context provenance includes only source event IDs owned by the requested user.

10. Persistent embedding pipeline

What is embedded

Only committed active profile facts are embedded. The canonical embedding text includes all structured identity fields:

category: dietary_preference
key: pasta_preference
value: "Makarna yemeyi çok severim."

Element	Implementation
Provider	Ollama /api/embed
Default model	embeddinggemma:latest
Dimensions	768, enforced in code and database constraint
Persistence	memory_embeddings with content_hash and model identity
Index	HNSW using vector_cosine_ops
Distance	pgvector cosine distance; similarity = 1 - distance
Query cache	Bounded process-local LRU, default 256 entries, keyed by model + SHA-256(text)
Durability	Embedding job committed in the same transaction as the profile fact

Why structured fact text is used

Embedding a normalized category/key/value representation gives the vector both semantic content and stable ontology hints. The raw conversation is not embedded as a long-term fact, which reduces noise and prevents temporary wording from polluting profile retrieval.

Failure behavior

If embeddings are disabled, absent, or unavailable, context construction continues with lexical and alias ranking. Missing vectors can be backfilled later. Deleting a profile fact cascades to its embedding and embedding job.

11. Retrieval algorithm

Profile retrieval stages

Signal	Implemented score/effect
Lexical overlap	Category up to 45 points, key up to 35, value up to 20, normalized by query token count.
Concept aliases	Up to 55 points for Turkish/English concepts such as hitap, kahve, ilaç, food, family.
Semantic similarity	+80 x cosine similarity when similarity ≥ 0.40 .
Pinned category	+100; defaults to communication and accessibility.
Recency	Up to +5 with decay over roughly 30 days.
Confidence	Up to +3.
Verified provenance	+4 for user_confirmed or caregiver_confirmed.
Protected-domain gate	Health/medication and emergency-contact profile facts require a matching query domain; value text alone cannot bypass it.
Bounds	At most 20 profile facts and 1500 estimated tokens by default; complete facts are omitted rather than truncated.

Candidate formation

A fact becomes a candidate when it has lexical overlap, alias overlap, semantic similarity above threshold, or a pinned category. Candidates are sorted by total score, timestamp, and ID, then packed into item/token budgets.

Other context sources

- Episodes: query-aware lexical/alias selection under their own item/token budget; no pgvector today.
- Temporary memories: newest active unexpired whole records, bounded by item and serialized-wrapper token budgets; not query-ranked today.
- Conversation: newest unexpired suffix under message/token bounds, returned chronologically.
- Temporary observations/session state: only unexpired state is included.

Why retrieval can be fast while chat is slow

context:build performs embedding + database search + deterministic packing. The optional chat path then runs a generative model over the prompt. In observed runs retrieval was tens of milliseconds, while local 12B response generation could take around ten seconds.

12. Context package consumed by the assistant

POST /v1/context:build returns an attributed, bounded package. The orchestrator sends only useful memory data to the reply model; retrieval debug metadata is not placed inside the model prompt.

```
{
  "as_of": "2026-09-23T08:11:02+00:00",
  "user_id": "demo-user-001",
  "session_id": "demo-session-001",
  "profile_facts": [...],
  "episodes": [...],
  "temporary_memories": [...],
  "temporary_observations": [...],
  "recent_messages": [...],
  "conversation_window": {...},
  "profile_retrieval": {...},
  "memory_refs": [...],
  "timing": {...}
}
```

Prompt assembly

- Input budget = CHAT_OLLAMA_NUM_CTX - CHAT_OLLAMA_NUM_PREDICT - 256 safety tokens.
- The current user text is never silently truncated.
- If necessary, observations, temporary items, session state, low-priority facts, and older history are removed until the prompt fits.
- Facts are wrapped with owner_user_id and provenance so first-person stored text is still attributed to the user.
- The system prompt states that the current explicit user correction overrides stale asynchronous memory.

Memory references and observability

memory_refs and episode_refs expose exactly which records entered the package. profile_retrieval.selected explains category, key, score, token cost, and reasons such as query_overlap, concept_alias, semantic_similarity, or pinned_category. timing separates query embedding, vector search, ranking, and assembly.

13. Safety, provenance, and emergency behavior

Evidence guards

- evidence_text must be a contiguous quote from the current user message.
- Numbers, clock times, weekday atoms, and temporal frequency are compared between evidence and extracted value.
- Question clauses, reported speech, unsupported subject attribution, fragments, and memory-control instructions are rejected.
- Model-produced string values that are not grounded are replaced by already-validated literal evidence in the default single-pass path.
- Financial identifiers and credentials are discarded from reusable memory.

Protected information

Health, emergency contact, and location can be stored automatically only as explicit user assertions. user_asserted means the user supplied the statement; it does not mean medically or externally verified. Existing protected values are masked as [protected] before they are sent back to the extraction model for slot matching.

Emergency orchestrator

Before normal reply generation, the optional conversation orchestrator evaluates high-signal current danger patterns: severe bleeding, breathing difficulty, loss of consciousness, chest pain, head impact/fall, post-fall neurological warning, stroke warning, seizure, inability to stand, and explicit requests for 112/ambulance. Up to four recent user messages may resolve a short follow-up.

Action result	Meaning
call_simulated	A usable user-asserted/verified contact with a valid Turkish mobile number was found. No real call is placed.
no_emergency_contact	Danger detected, but no actionable contact was found; user is advised to call 112.
already_active	An emergency action was already initiated in this session.

Production warning

This is not a certified medical or emergency system. Real calling, device control, escalation policy, authentication, monitoring, encryption, retention, consent, and regional compliance remain external production work.

14. API surface

Method	Endpoint	Purpose
GET	/healthz	Service health.
GET	/v1/analyzer/status	Extractor/model readiness.
GET	/v1/embeddings/status	Profile embedding coverage and job counts.
POST	/v1/interactions:enqueue	Recommended durable asynchronous ingestion.
GET	/v1/interactions/{event_id}/status	Outbox status and eventual decisions.
POST	/v1/interactions:process	Synchronous diagnostic processing.
POST	/v1/context:build	Query-aware bounded context package.
POST / GET	/v1/sessions/{session_id}/messages	Append/list TTL conversation history.
POST	/v1/temporary-memories	Trusted explicit short-term write.
GET	/v1/sessions/{session_id}/temporary-memories	List active short-term items.
POST	/v1/memories	Trusted explicit profile write.
GET	/v1/users/{user_id}/memories	List active or historical profile facts.
GET	/v1/users/{user_id}/episodes	List active-retention episodes.
POST	/v1/events	Append an explicit idempotent event.
PUT	/v1/sessions/{session_id}/state	Replace explicit session state.
GET	/v1/candidates	Inspect extraction audit decisions.
DELETE	/v1/memories/{memory_id}	Delete one owned profile fact and cascading vector.
DELETE	/v1/episodes/{episode_id}	Delete one owned episode.
DELETE	/v1/sessions/{session_id}	Delete session state/history/temporary memory and cancel unfinished jobs.

Swagger UI

<http://127.0.0.1:8001/docs>

15. Installation and one-command operation

Prerequisites

- Python 3.11+ (the current workspace uses Python 3.14).
- PostgreSQL with the pgvector extension.
- Ollama with gemma4:12b and embeddinggemma:latest installed.

First-time setup

```
cd /Users/egehankaraca/Desktop/llm_memory
python3 -m venv venv
venv/bin/pip install -r requirements.txt

createdb memory_db
ollama pull gemma4:12b
ollama pull embeddinggemma:latest

export DATABASE_URL='postgresql+psycopg2:///memory_db'
venv/bin/alembic upgrade head
```

Recommended memory-only demo

```
venv/bin/python scripts/run_demo.py \
--model gemma4:12b \
--database-url 'postgresql+psycopg2:///memory_db' \
--user-id 'demo-user-001' \
--session-id 'demo-session-001' \
--debug
```

The launcher checks Ollama models, applies migrations, chooses a free API port, starts FastAPI and the async worker, then opens a single-terminal interface.

Optional real chat mode

```
venv/bin/python scripts/run_demo.py \
--chat \
--model gemma4:12b \
--database-url 'postgresql+psycopg2:///memory_db' \
--user-id 'demo-user-001' \
--session-id 'demo-session-001' \
--debug
```

Reuse both IDs to continue the same profile and short-term session. Reusing only user_id starts a fresh session while retaining long-term profile facts and user-scoped episodes.

16. How to use and test the system

Memory-only commands

Command	Expected view
<code>/profile</code>	Active long-term facts.
<code>/session</code>	Active unexpired temporary memories.
<code>/episodes</code>	User episodic history.
<code>/debug</code>	Toggle technical context and decision details.
<code>/help</code>	Show commands.
<code>/exit</code>	Stop UI, worker, and API started by the launcher.

Feature-covering manual sequence

Input	Expected memory behavior
Bana bundan sonra Ahmet Bey diye hitap et.	profile: communication.form_of_address
Her sabah sade Türk kahvesi içerim.	profile: routine/preference slot
Makarna yemeyi çok severim.	separate profile food preference; must not replace coffee
Tansiyon ilacımı her sabah saat 8'de alırım.	health-sensitive profile, verification=user_asserted
Acil durumda kızım Ayşe'yi 0555 123 45 67 numarasından ara.	emergency_contact profile; valid phone becomes actionable
Bugün öğleden sonra yürüyüşe çıkmak istiyorum.	session memory; expires at local day end
Bu akşam haberleri izlemek istiyorum.	independent session memory
Şimdi biraz dinleneceğim.	current-session temporary memory
Dün banyoda düştüm.	episode: health_event.fall_incident
Bugün hava nasıl?	discard/ignored; no reusable memory
tenis	discard/ignored fragment
İnternet şifrem abc123.	credential discarded
Düzeltilme: Her sabah sütlü Türk kahvesi içerim.	supersede coffee slot only

Retrieval checks

```

/context Bana nasıl hitap etmelisin?
/context Sabahları ne içerim?
/context Acıktım, ne yemek önerirsin?
/context Tansiyon ilacımı kaçta alıyorum?
/context Bugünkü geçici planlarım neler?
/context Dün ne olmuştu?

```

In the one-command memory-only UI the equivalent visible commands are `/profile`, `/session`, and `/episodes`. For a raw context request use the advanced terminal demo or Swagger/curl endpoint.

17. Performance and evaluation evidence

Retrieval benchmark - p50 latency (milliseconds)



Source: `evals/retrieval_benchmark.json`, PostgreSQL `pgvector` 0.8.6, `top-k` 3.

Interpretation

- At 10K vectors, the benchmark's forced HNSW path had p50 1.908 ms and 0.94 recall@3.
- Exact 10K cosine scan had p50 31.930 ms and 1.0 recall@3.
- Uncached Ollama query embedding had p50 33.840 ms after the first request.
- Observed application context builds were commonly about 20-50 ms for small real profiles.
- A local gemma4:12b chat answer may still take around ten seconds; this is reply-model generation, not database retrieval.

Current automated verification

The repository's unittest suite was executed while producing this guide: 310 tests passed in 1.089 seconds. The tests use isolated SQLite and mocked model calls where appropriate; real Ollama quality is measured separately by JSONL evaluation runners.

Model evaluation records

The eval directory preserves historical experiments rather than one universal current score. Reports include 4B/8B comparisons, stability runs, memory-scope runs, and retrieval benchmarks. The most recent stored full scope report contains 50 cases with 47 passing (94%), while targeted follow-up reports contain the repaired cases. These model-era results should not be presented as a current end-to-end production SLA.

Measure layers separately

Track enqueue latency, worker completion time, query embedding time, vector search time, total context-build time, and reply-model time independently. A slow extractor or reply model should not be misdiagnosed as a pgvector problem.

18. Configuration reference

Variable	Default	Purpose
DATABASE_URL	postgresql+psycpg2:///memory_db	Shared database for migrations, API, and worker.
MEMORY_ANALYZER_PROVIDER	ollama	Semantic extractor or rules fallback mode.
OLLAMA_MODEL	gemma4:12b	Memory extraction model; one-command demo also assigns it to chat.
MEMORY_PROFILE_EMBEDDING_MODEL	embeddinggemma:latest	768-dimensional embedding model.
MEMORY_ASYNC_INGESTION	true	Use outbox fast path.
MEMORY_OUTBOX_MAX_ATTEMPTS	5	Extraction retry budget.
MEMORY_WINDOW_MAX_MESSAGES / TOKENS	10 / 2000	Sliding conversation limits.
MEMORY_CONVERSATION_TTL_HOURS	24	Conversation read lifetime.
MEMORY_TEMPORARY_MAX_ITEMS / TOKENS	20 / 1500	Temporary context bounds.
MEMORY_TEMPORARY_TTL_MINUTES	60	Default current-session TTL.
MEMORY_PROFILE_MAX_FACTS / TOKENS	20 / 1500	Profile context bounds.
MEMORY_PROFILE_PINNED_CATEGORIES	communication,accessibility	Always-considered non-protected categories.
MEMORY_EPISODE_MAX_ITEMS / TOKENS	10 / 1000	Episode context bounds.
MEMORY_PROFILE_SEMANTIC_MIN_SIMILARITY	0.40	Semantic candidate threshold.
MEMORY_PROFILE_VECTOR_TOP_K	20	Maximum pgvector neighbors.
CHAT_OLLAMA_NUM_CTX / NUM_PREDICT	4096 / 512	Reply prompt and output budget.

19. Known limitations and production gaps

Area	Current limitation	Production direction
Authentication	Caller supplies user_id; no real auth layer.	Add identity, authorization, tenancy, and rate limits before network exposure.
Privacy	No application-level field encryption; raw events/audits can contain personal data.	Encryption, secrets management, restricted DB access, retention and deletion policy.
Emergency	Pattern detector and simulated call only.	Safety-reviewed tool integration, monitoring, escalation policy, compliance.
Episodes	No semantic vectors and no duplicate consolidation.	Optional episode embedding/deduplication if real usage requires it.
Temporary retrieval	Newest active items are bounded but not query-ranked.	Add query-aware session ranking only if profiles become noisy.
TTL cleanup	Expired rows are filtered, not guaranteed to be physically deleted at expiry.	Scheduled retention/purge job.
Relative event time	Episode occurred_at is ingestion time; 'dün' stays in text.	Temporal normalization with provenance if timeline accuracy is required.
Model quality	Local reply wording and extraction can vary by model/hardware.	Maintain separate extraction evals, chat acceptance review, and latency SLOs.
External tools	No live weather, device control, STT, or TTS.	Integrate via the real assistant orchestrator outside the Memory Service.

Demo readiness

For a memory-management demo, the implemented system covers classification, all four scopes, storage, supersession, TTL, episodic recall, hybrid retrieval, async ingestion, embeddings, observability, and deletion. Production readiness requires the controls above.

20. Repository map and inspection basis

This guide was prepared from the complete non-virtual-environment repository inventory: application modules, Alembic configuration and all 13 migrations, all demo/evaluation/benchmark scripts, every test module and test name, all JSONL datasets, and all stored JSON report summaries. Generated bytecode and platform metadata were not treated as source.

Area	Primary files
API and database	main.py, database.py, models.py
Classification/policy	memory_analyzer.py, memory_safety.py
Conflict handling	memory_consolidator.py
Retrieval	memory_retriever.py, profile_semantic.py, memory_embeddings.py
Short-term	temporary_memory.py
Async	memory_outbox.py, memory_worker.py
Optional assistant	conversation_orchestrator.py, emergency_orchestrator.py
Schema	alembic/env.py and alembic/versions/20260910_01 through 20260918_13
Operations	scripts/run_demo.py, run_memory_terminal_demo.py, run_chat_demo.py, run_memory_worker.py, backfill_memory_embeddings.py
Evaluation	run_memory_evals.py, run_memory_scenarios.py, run_retrieval_benchmark.py, run_async_memory_benchmark.py and evals/*
Tests	tests/test_*.py - 310 tests passing during document generation
Configuration/docs	.env.example, requirements.txt, alembic.ini, README.md

Recommended demo narrative

Show one stable preference, one health routine, one temporary plan, and one episode. Then demonstrate correction/supersession, retrieval from a fresh session, the context timing block, and the profile/session/episode views. Keep optional chat quality explicitly separate from memory correctness.

Final mental model

Store facts by lifetime; retrieve by relevance; generate language last.

The memory model's responsibility is to preserve attributed user information safely and return only the bounded evidence needed for the current query. The reply model remains replaceable.